DistilBERT and ALBERT

1.Background: BERT

- Bidirectional encoder representations from transformers (BERT) is a Language Model. It learns to represent text as a sequence of vectors using self-supervised learning. It uses the encoder only transformer architecture.
- BERT is trained by masked token prediction and next sentence prediction. With this training, BERT learns contextual, latent representations of tokens in their context.

2.DistilBERT

DistilBERT (Distilled BERT) is a compressed version of BERT created using knowledge distillation a technique where a smaller "student" model learns from a larger "teacher" model's output.

It is pretrained by knowledge distillation to create a smaller model with faster inference and requires less compute to train.

Why it was made:

BERT is powerful but *big and slow*. DistilBERT aims to:

- Reduce model size (~40% smaller than BERT)
- Maintain high quality (≈97% of BERT performance)
- Speed up inference (≈60% faster)

What is Knowledge distillation:

- Knowledge distillation is a model compression technique in which a smaller "student" model is trained to mimic the output distributions (soft targets) of a larger "teacher" model, rather than learning solely from hard labels or one-hot encoded gold classes.
- The student model is guided to approximate the full output distributions of the teacher, capturing the uncertainty and "dark knowledge" present in the teacher's predictions.
- The distillation process typically involves minimizing loss functions such as Kullback Leibler divergence between the student and teacher outputs.

DistilBERT employs a specific distillation process that uses a triple loss function, combining distillation loss, masked language modeling loss, and cosine embedding loss. The distillation loss encourages the student model to match the output probabilities of the teacher, the masked language modeling loss is used for language modeling, and the cosine embedding loss aligns the representations between student and teacher.

Training DistilBERT:

DistilBERT is trained using a **triple loss function** which combines:

1. **Language Modeling Loss**:

   This is the standard BERT training objective:

   - Random words in a sentence are masked
   - The model predicts the original word

   It Ensures DistilBERT actually learns language and Prevents it from just copying BERT blindly.

2. **Distillation Loss**:

   Instead of learning only from the correct label, DistilBERT learns from BERT's full probability distribution.

   Example:

   BERT output:

   "great" = 0.60, "good" =0.25,"nice"= 0.10, "bad"=0.05

   DistilBERT tries to match this entire distribution, not just "great".

   - These are called **soft labels**
   - They encode *relationships between words*
   - The student learns why one word is more likely than another

3. **Cosine-Distance Loss**:

   This loss aligns the hidden states (internal embeddings) of DistilBERT with BERT.

   "Not only predict like BERT think like BERT."

   - Take hidden layer vectors from teacher and student

- Measure the **cosine similarity** between them
- Minimize the cosine distance

Helps the student learn semantic structure and Makes representations more BERT-like.

By combining these losses DistilBERT is able to learn efficiently from BERT while maintaining high performance.

Imagine learning from a professor:

- Language Modeling Loss → You study the textbook
- Distillation Loss → You copy how the professor answers questions
- Cosine Loss → You adopt the professor's way of thinking

That's why DistilBERT keeps 97% of BERT's performance with much fewer parameters.

ALBERT

ALBERT stands for A Lite BERT, another BERT variant that aims to improve efficiency without lowering performance.

Instead of shrinking the model via knowledge distillation, ALBERT:

- Shares parameters across layers reducing total trainable weights.
- Uses a different auxiliary objective (Sentence Order Prediction, SOP) to improve training.

ALBERT keeps the same architecture depth as BERT but drastically reduces the number of parameters. It does this using two main innovations:

1. **Factorized embedding parameterization**
2. **Cross-layer parameter sharing**
3. Plus a training improvement:
   Sentence Order Prediction (SOP)

1) Factorized embedding parameterization:

The problem in BERT:

- Vocabulary embedding size = hidden size

- Example:
    - Vocab size = 30,000
    - Hidden size = 768
    - Embedding parameters = 30,000 × 768 ≈ 23M parameters That's huge.
- Vocab → 768 → Transformer

ALBERT splits the embedding size from the hidden size:

Small embedding layer:

- Each word is represented as a smaller vector (128 instead of 768)
- This drastically reduces the number of embedding parameters:

$$30{,}000 \times 128 \approx 3.84 \text{ million parameters (vs 23M)}$$

Projection layer

- The small embedding is multiplied by a learnable projection matrix to convert it to the Transformer hidden size (768).

Vocab → 128 → projection → 768 → Transformer

2) Cross-Layer Parameter Sharing:

What happens in BERT Each transformer layer has:

- Its own attention weights
- Its own feed-forward weights

So 12 layers = 12 completely separate parameter sets.
Think of it like this:

- BERT:
  Layer 1 = different brain
  Layer 2 = different brain
- ALBERT:
  All layers = same brain, reused

ALBERT shares the same parameters across all layers, The data changes as it flows through layers, but the weights stay the same.

3) **Sentence Order Prediction (SOP):**

BERT uses Next Sentence Prediction (NSP) as one of its training tasks:

- Randomly pairs two sentences:
  - 50% chance: Sentence B actually follows Sentence A (positive example)
  - 50% chance: Sentence B is a random sentence from the corpus (negative example)
- The model predicts: "Does B follow A?"

Random negative sentences often come from different topics, so the model can just learn:

"Do these two sentences talk about the same topic?" It doesn't really learn sentence order or coherence.

ALBERT replaces NSP with **Sentence Order Prediction (SOP)**:

- Instead of pairing random sentences, use two consecutive sentences from the corpus
- Then create two versions:
  1. **Correct order**: A → B
  2. **Swapped order**: B → A

The model predicts: "Are these sentences in the correct order or swapped?"

SOP predicts actual sentence order.

Mathematical formula:

- SA,SB = embeddings of the two sentences.
- Model outputs probability $p = \text{softmax}(f([S_A; S_B]))$
- Where f is the classifier head predicting:
- $\hat{y} = \begin{cases} 1 & \text{if correct order} \\ 0 & \text{if swapped} \end{cases}$
- Loss = **binary cross-entropy** between predicted probability and true label:
  LSOP=−ylog(p)−(1−y)log(1−p)
- During training, SOP loss is combined with Masked Language Modeling (MLM) loss:

$$\mathcal{L}_{total} = \mathcal{L}_{MLM} + \mathcal{L}_{SOP}$$

SOP teaches the model to "think logically like a human," rather than just recognize topic similarity.

DistilBERT vs ALBERT

- DistilBERT is ideal when fast inference and smaller models are needed, leveraging knowledge distillation to learn from BERT.
- ALBERT is ideal when memory efficiency and deep architectures are required, leveraging parameter sharing and factorized embeddings to reduce model size without sacrificing depth.

DistilBERT → “Learn from a teacher and summarize knowledge” while ALBERT → “Reuse the same brain efficiently while thinking logically”.

Both models demonstrate that BERT-level performance can be maintained with far fewer resources, but the choice depends on whether speed (DistilBERT) or memory efficiency and deep modeling (ALBERT) is more important for the task.